

Simple EMI / Loan Calculator

Submitted By:

1. Ghilman Shad (24P-3101)
2. Ghulam Muhammad (24P-3026)

Course Name:

Introduction to Software Engineering

Course Instructor:

Muhammad Umer Haroon

Institution:

National University of Computer and Emerging Sciences, Peshawar Campus

Date of Submission:

26th November, 2025

1. Project Overview:

This project involved the modernization and refactoring of a legacy C++ financial calculation codebase. The original system was a monolithic, console-based application plagued by architectural issues, lack of input validation, and potential integer overflow risks.

Our team successfully refactored the application into a modular, object-oriented architecture. We implemented robust input validation, upgraded the mathematical precision to long double, and introduced a configuration file system to eliminate hardcoded values. Furthermore, we established a rigorous quality assurance process using GoogleTest for unit testing and Doxygen for automated documentation.

Key Deliverables:

Modular Codebase: Separation of concerns into `Loan.h` (Interface), `Loan.cpp` (Implementation), and `main.cpp` (Driver).

Enhanced Precision: Transitioned from standard floating-point types to long double to ensure accuracy in large financial computations.

Input Integrity: Comprehensive validation logic to prevent invalid financial states (e.g., negative principal or interest).

Automated Testing: Implementation of a GoogleTest suite covering edge cases and standard operations.

2. Refactoring & Architecture:

The legacy `main.cpp` file contained class definitions, logic, and user interface code in a single scope. We decoupled this into a standard C++ header/implementation pattern:

`src/Loan.h`: Contains the class definition and Doxygen documentation. It serves as the "contract" for the module.

`src/Loan.cpp`: Contains the business logic and mathematical formulas. This encapsulates the implementation details.

`src/main.cpp`: Acts as the entry point, handling argument parsing and configuration loading.

Code Snippet: Header File Separation (Loan.h)

```
class LoanCalculator {
public:
    LoanCalculator();
    // Logic removed from header to improve encapsulation
    void setAmount(long double A);
    void setInterest(long double i);
    // ...
    long double calculatePayment();
private:
    long double amount_;
    long double interest_;
    // ...
};
```

3. Bug Fixes & Technical Improvements:

Bug Fix 1: Input Validation

Problem: The legacy code allowed users to input negative loan amounts or interest rates, leading to nonsensical financial outputs.

Solution: We moved setter functions to Loan.cpp and added strict validation logic.

Code Snippet (Loan.cpp):

```
void LoanCalculator::setAmount(long double A) {
    if (A <= 0) {
        // Validation: Prevent negative inputs
        amount_ = 1.0; // Fail-safe default
    } else {
        amount_ = A;
    }
    amountSet_ = true;
}
```

Bug Fix 2: Precision & Overflow Prevention

```
0
1  long double calculateLoanBalance();
2  long double calculatePayment();
3  long double calculateNumberPayments();
4  long double calculateLoanAmount();
5  long double calculateInterestRate();
6  long double calculateEffectiveInterestRate();
7
8  std::string toString();
9
10 private:
11  long double amount_;
12  bool amountSet_;
13
14  long double initialPayment_;
15
16  long double interest_;
17  long double interestPeriodic_;
18  bool interestSet_;
19
20  long double payment_;
21  bool paymentSet_;
22
```

Improvement 3: Dynamic Configuration

Problem: Default values were hardcoded, requiring recompilation to change.

Solution: Implemented a parser for config.txt to load defaults at runtime.

```
7  * @brief Explaining readConfig from reading values with file handling from config.txt
8  */
9  void readConfig(long double &amount, long double &rate, int &months) {
10     std::ifstream file("config.txt");
11
12     if (!file.is_open()) {
13         std::cout << "config.txt not found. Using defaults.\n";
14         return;
15     }
16
17     std::string name;
18     std::string value;
19
20     // Each line should look like: Name=Value
21     while (std::getline(file, name, '=') && std::getline(file, value)) {
22         if (name == "Amount") {
23             amount = std::stold(value);
24         }
25         else if (name == "Interest") {
26             rate = std::stold(value);
27         }
28         else if (name == "Years") {
29             months = std::stoi(value) * 12;
30         }
31     }
32
33     std::cout << "Configuration loaded.\n";
34 }
35
36 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
```

4. Quality Assurance (Unit Testing):

We utilized the GoogleTest framework (v1.12.1) to validate the system. The library was included locally in lib/ to ensure portability across development environments.

Test Coverage:

Standard EMI: Verifies the math against known financial examples.

Negative Input: Ensures the system rejects invalid inputs gracefully.

Large Tenure: Stress-tests the long double implementation with large values.

Test Execution Output:

```
ghulam-muhammad@gmspc:~/legacy-calc-2009$ ./runTests
[=====] Running 3 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 3 tests from LoanTest
[ RUN      ] LoanTest.CalculatesStandardEMI
[ OK       ] LoanTest.CalculatesStandardEMI (0 ms)
[ RUN      ] LoanTest.HandlesNegativeInput
[ OK       ] LoanTest.HandlesNegativeInput (0 ms)
[ RUN      ] LoanTest.HandlesLargeTenure
[ OK       ] LoanTest.HandlesLargeTenure (0 ms)
[-----] 3 tests from LoanTest (0 ms total)

[-----] Global test environment tear-down
[=====] 3 tests from 1 test suite ran. (0 ms total)
[ PASSED  ] 3 tests.
ghulam-muhammad@gmspc:~/legacy-calc-2009$
```

```
ghilman@fedora:~/legacy-calc-2009$ ./runTests
[=====] Running 3 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 3 tests from LoanTest
[ RUN      ] LoanTest.CalculatesStandardEMI
[ OK       ] LoanTest.CalculatesStandardEMI (0 ms)
[ RUN      ] LoanTest.HandlesNegativeInput
[ OK       ] LoanTest.HandlesNegativeInput (0 ms)
[ RUN      ] LoanTest.HandlesLargeTenure
[ OK       ] LoanTest.HandlesLargeTenure (0 ms)
[-----] 3 tests from LoanTest (0 ms total)

[-----] Global test environment tear-down
[=====] 3 tests from 1 test suite ran. (0 ms total)
[ PASSED  ] 3 tests.
```

5. Git Commit History:

The project was managed using Git with a clear separation of features. Below is a summary of our development timeline.

GHULAM MUHAMMAD's COMMIT HISTORY

commit 29489c5da4c7381bf08da323d0759ab592637613 (HEAD -> dev)

Author: GM-UNAR <gmuhd110@gmail.com>

Date: Wed Nov 26 04:39:05 2025 +0500

Fixed google test without nested repo

commit a99fdbd8eb14ee52b3c1b8b297fb459cb8ef7ac9

Author: GM-UNAR <gmuhd110@gmail.com>

Date: Wed Nov 26 02:46:19 2025 +0500

GHULMAN COUNTINUED DOXYGEN DOCUMENTATION ABOUT THE CONFIG FILE AND readConfig FUNTION

commit 83aa379be9a7ff58d13419ba10e52030b4c302af

Author: GM-UNAR <gmuhd110@gmail.com>

Date: Wed Nov 26 02:36:43 2025 +0500

GHULAM MUHAMMAD ADDED CONFIG.TXT TO READ VALUES , PUT THE LOGIC IN THE MAIN

commit d0668cbc9e6fc433957a060c3ba3f906481f270b

Author: GM-UNAR <gmuhd110@gmail.com>

Date: Wed Nov 26 02:24:01 2025 +0500

GHULAM MUHAMMAD ADDED DOXYGEN COMMENTS FOR THE PREVOIUS COMMIT AND RENAMED FILES

commit 105bd0c0bea97418eb43290e4bd395d3ae3938bc

Author: GM-UNAR <gmuhd110@gmail.com>

Date: Wed Nov 26 02:08:07 2025 +0500

GHILMAN: HANDLED DATAPYED OVERFLOW , MODIFIED FUNTIONS , hDEALT WITH HEADERFILES

commit 0a6667dcc991bdfd79a3dfd58d94cc3d1ef83412 (origin/dev)

Author: GM-UNAR <gmuhd110@gmail.com>

Date: Tue Nov 25 22:38:17 2025 +0500

GHULAM MUHAMAMD Setup (src, test , docs)

commit 347e865b6f0e3ab8105ff2d5e3110882d012a476 (origin/master, origin/HEAD, master)

Author: Brady Johnson <bradyallenjohnson@gmail.com>

Date: Wed Apr 18 11:32:49 2012 +0300

Added compilation information

commit dff0427045abc9605c45e5a3425946d63d4cae00

Author: Brady Johnson <brady.allen.johnson@ericsson.com>

Date: Wed Apr 18 04:27:20 2012 -0400

Nicer formatting on Readme and added better app help text

commit 71f0456d9218af942802b4c2718c88922a6d5495

Author: Brady Johnson <brady.allen.johnson@ericsson.com>

Date: Wed Apr 18 04:21:29 2012 -0400

Added readme and additional app help text

:

GHILMAN ALI'S COMMIT HISTORY:

commit ae802616532de22d40d2a9524652731414799f25 (HEAD -> dev)

Author: Ghilman Shad <ghilmanshad@gmail.com>

Date: Wed Nov 26 04:39:16 2025 +0500

Fixed GoogleTest without repo

commit f6b855b9c9cdd7df6bf0922b3329d7c35b9dfad3

Author: Ghilman Shad <ghilmanshad@gmail.com>

Date: Wed Nov 26 02:46:23 2025 +0500

Ghilman continued doxygen documentation about the config.txt and readconfig function

commit 015a35afa58e66c57f270f201bb6ba2788ac4f91

Author: Ghilman Shad <ghilmanshad@gmail.com>

Date: Wed Nov 26 02:36:48 2025 +0500

Ghulam Muhammad added config.txt file to read values, insert the logic in the main

commit 270cd866c82e144e96527f71cc597be95ed47655

Author: Ghilman Shad <ghilmanshad@gmail.com>

Date: Wed Nov 26 02:24:04 2025 +0500

Ghulam Muhammad added doxygen comments for the previous commit and renamed the files

commit faf1461d881210cfe9f2484bd1e0cc5d50dde388

Author: Ghilman Shad <ghilmanshad@gmail.com>

Date: Wed Nov 26 02:02:37 2025 +0500

GHILMAN: HANDLED DATAPYED OVERFLOW , MODIFIED FUNCTIONS , hDEALT WITH HEADERFILES

commit e086f654c7f6b8b71edd4ed5b46f532e508b73cb (origin/dev)

Author: Ghilman Shad <ghilmanshad@gmail.com>

Date: Tue Nov 25 23:06:09 2025 +0500

Ghilman Setup .gitignore Build Artifacts

commit 347e865b6f0e3ab8105ff2d5e3110882d012a476 (origin/master, origin/HEAD, master)

Author: Brady Johnson <bradyallenjohnson@gmail.com>

Date: Wed Apr 18 11:32:49 2012 +0300

Added compilation information

commit dff0427045abc9605c45e5a3425946d63d4cae00

Author: Brady Johnson <brady.allen.johnson@ericsson.com>

Date: Wed Apr 18 04:27:20 2012 -0400

:|

6. Conclusion:

We both have given our best and put all of it in doing this assignment and finally the old legacy code is now the a robust, maintainable, and verifiable software product. By adhering to Separation of Concerns and implementing automated testing, we have eliminated the technical debt of the legacy codebase and prepared it for future scalability.